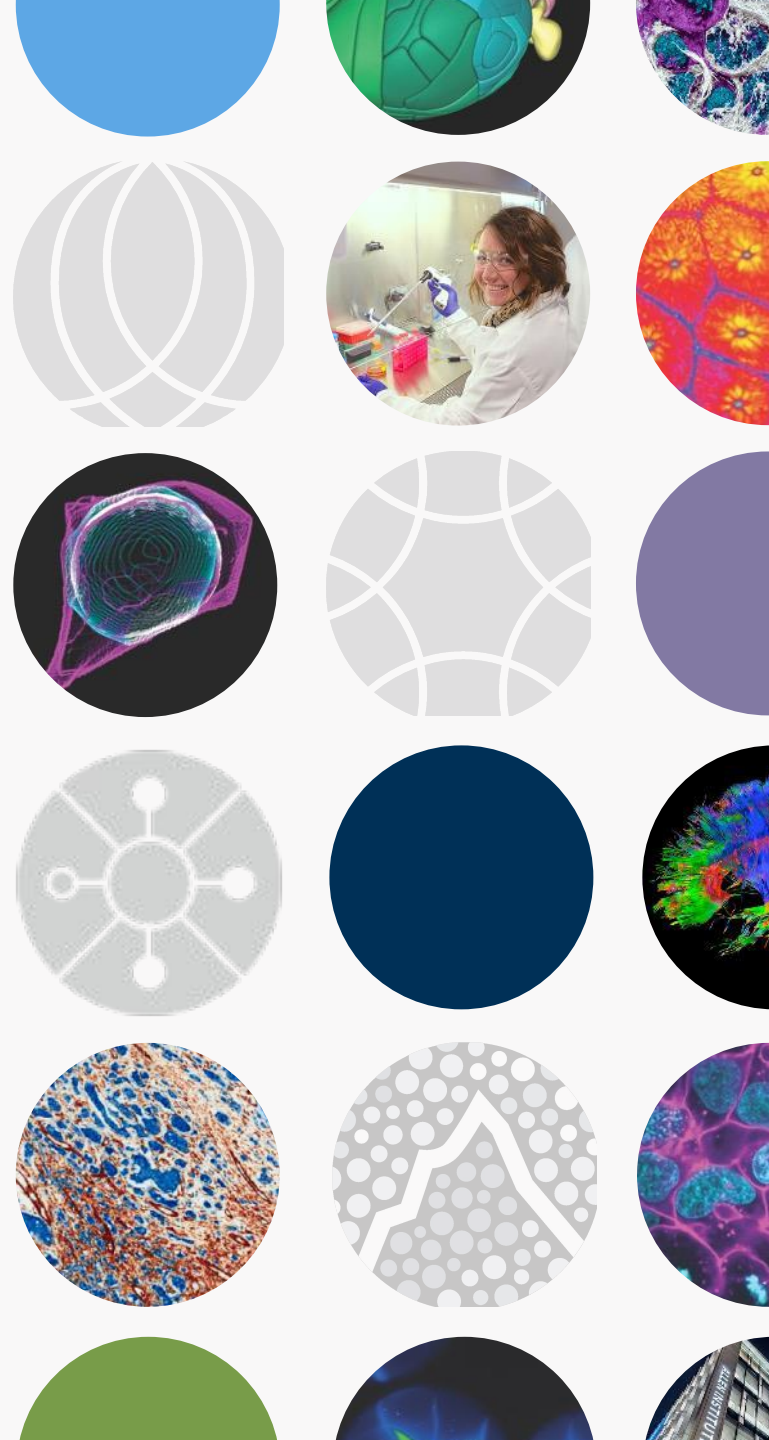




Packages & NumPy



Today's agenda

Group
dinner at
5:30p!

➔ Breakfast + Questions

9:00 AM - 10:00 AM

Basic Python 2: Control Flow & Functions (Ashley)

10:00 AM - 11:00 AM

Intro to Packages (Ashley)

11:00 AM - 12:00 PM

Lunch

12:00 PM - 1:00 PM

Pedagogical Unpacking (Rachel)

1:00 PM - 1:30 PM

Intro to NumPy + Pandas (Ashley + Theresa)

1:30 PM - 2:30 PM

Afternoon Break

2:30 PM - 3:00 PM

Intro to Matplotlib (Rachel) (+ Patch Seq?)

3:00 PM - 4:30 PM

End of Day Reflections (Rachel)

4:30 PM - 5:00 PM

Packages in Python

Python's standard library works for some purposes, but there are many very useful packages for additional purposes:

- **numpy** (<http://numpy.scipy.org>): numerical Python
- **scipy** (<http://www.scipy.org>): scientific Python; built on numpy
- **matplotlib** (<http://www.matplotlib.org>) graphics library



Name A Better Trio. I'll Wait 🙄



```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
```

That's it.

[Source](#)

Installing packages & importing modules

To install packages, use

```
$ pip install PACKAGE
```

You can then import modules from the package with

```
>>> from PACKAGE import MODULE
```

to see all of the modules available, use

```
>>> print(dir(MODULE))
```

We typically won't need to do this in the Colab, because many packages are already installed. However, you *may* need to do this for local notebook operation, or for special notebooks (like allensdk!)

Module	Built-In	Description
csv	Yes	Aids in the reading, writing, and analysis of CSV files.
zipfile	Yes	Aids in the creation and extraction of compressed ZIP archive files.
matplotlib	No	Graphics library for plotting
plotly	No	A graphics library used for creating interactive plots for the web.
seaborn	No	A graphics library built on top of matplotlib with high-quality plots
pandas	No	A data processing library that specializes in data frames, which are analogous to spreadsheets.
scikit-learn	No	Contains basic tools for machine learning (i.e., helping to learn from data and make predictions).
numpy	No	Offers highly efficient data processing.
pygame	No	A game programming library that helps to build interactive, graphical games in Python.
django	No	Web development library that aids in designing websites and web applications.

Common Python modules — ones we'll work with are highlighted

We're learning how to deal with more and more complex data

```
data_point = 8.02
```



single variable
(int, float,
string)

```
data = [8.38, 3.34, 6.35]
```



data structure
(list, tuple,
dictionary)

```
big_data = [data_1, data_2, ...]
```



array
or dataframe

NumPy is the fundamental package for scientific computing with Python

- A numpy array is a grid of values which are all the same type (they're homogenous)
- Useful attributes:
 - `ndim` = # of dimensions
 - `shape` = a tuple of integers giving the size of the array along each dimension
 - `dtype` = type of data

Numpy Arrays

my_array = 1D array

3	2	4	1
---	---	---	---

```
my_array[0] = 3  
my_array.ndim = 1  
my_array.shape = (4,)  
my_array.size = 4
```

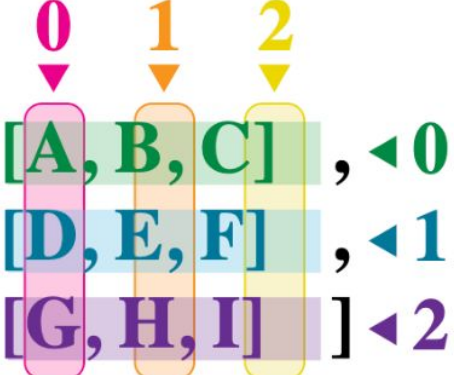
2D array

3	2	4	1
1	2	5	3

how to index
2D NumPy
arrays



```
my_array[1,3] = 3  
my_array.ndim = 2  
my_array.shape = (2,4)  
my_array.size = 8
```

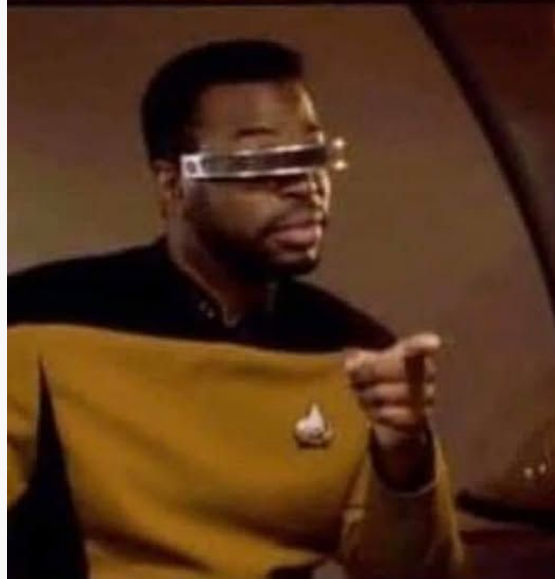

`data = [ [A, B, C], [D, E, F], [G, H, I] ]`
`data[ 0, 0] = A` `data[ 0, 1] = B` `data[ 0, 2] = C`
`data[ 1, 0] = D` `data[ 1, 1] = E` `data[ 1, 2] = F`
`data[ 2, 0] = G` `data[ 2, 1] = H` `data[ 2, 2] = I`

Indexing numpy arrays

Image from [Programming with Python: Analyzing Patient Data](#)



learning numpy axis rules



**print output
array's
shape until
one of the
the axis
values
works out**

Slicing & indexing NumPy arrays works *almost* the same as with Python lists

However, be aware that if you slice an array, it changes the original array.

If you need to copy, you need to explicitly do:

```
v3 = v[2:4].copy()
```

In this case, we would not change original array (v).

```
In [33]: v = np.random.random((5,4))  
v
```

```
Out[33]: array([[0.70782755, 0.1080363 , 0.63931318, 0.30594658],  
                [0.23089631, 0.58842692, 0.03879193, 0.56396161],  
                [0.92250973, 0.54564224, 0.89690301, 0.76679512],  
                [0.83668402, 0.18075749, 0.54652922, 0.03487156],  
                [0.48236452, 0.77258043, 0.61857768, 0.66614441]])
```

```
In [35]: v2 = v[2:4]  
v2
```

```
Out[35]: array([[0.92250973, 0.54564224, 0.89690301, 0.76679512],  
                [0.83668402, 0.18075749, 0.54652922, 0.03487156]])
```

```
In [37]: v2[1,3] = 2
```

```
In [38]: v
```

```
Out[38]: array([[0.70782755, 0.1080363 , 0.63931318, 0.30594658],  
                [0.23089631, 0.58842692, 0.03879193, 0.56396161],  
                [0.92250973, 0.54564224, 0.89690301, 0.76679512],  
                [0.83668402, 0.18075749, 0.54652922, 2.          ],  
                [0.48236452, 0.77258043, 0.61857768, 0.66614441]])
```

You can also use lists & booleans to index NumPy arrays

`my_array[[1,2,3]]`

`my_array[my_array > 1]`

We can also use this to selectively operate on values in the array that meet our criteria:

`my_array[my_array > 1] = my_array[my_array > 1] * 2`

Useful NumPy functions

`np.zeros()`

`np.empty()`

`np.linspace()`

`np.arange()`

`np.reshape()`

`np.random.random()`

`np.vstack()`

`np.hstack()`

`np.save()`

`np.load()`

See [here](#) for a useful Numpy overview.

Key NumPy takeaways

- Import a library into a program using `import libraryname`
- Use the NumPy library to work with arrays in Python.
- The expression `array.shape` gives the shape of an array.
- Use `array[x, y]` to select a single element from a 2D array.
- Array indices start at 0, not 1.
- Use `low:high` to specify a slice that includes the indices from low to high-1.
- Use `np.mean(array)`, `np.max(array)`, and `np.min(array)` to calculate simple statistics.
- Use `np.mean(array, axis=0)` or `np.mean(array, axis=1)` to calculate statistics across the specified axis.